

5

FORM 2

THE PATENT ACT 1970

(39 of 1970)

&

The Patents Rules, 2003

10

COMPLETE SPECIFICATION

(See section 10 and rule 13)

1. TITLE OF THE INVENTION:

15

**“System and Method for AI-Driven Energy-Efficient Workload Optimization
in Distributed Computing Systems”**

2. APPLICANT(S):

20

Name : STRATEGEMIST GLOBAL PRIVATE LIMITED

**Address : Plot 245, S.No 28, Sai Aishwarya Colony, Medipalli,
Ghatkesar, Hyderabad- 500098, Telangana, India.**

Nationality : Indian Company

25

30

3. PREAMBLE TO THE DESCRIPTION

The following specification particularly describes the invention and the manner in which it is to be performed.

35

4. DESCRIPTION:

Field of Invention

The present invention relates to an AI-driven system and method for energy-efficient workload execution in distributed computing environments. More particularly, the invention pertains to techniques for dynamically managing computational tasks and optimizing resource allocation based on real-time power availability and system constraints.

Background of the Invention

[0001] The present invention relates to an energy-efficient computing framework employing artificial intelligence-driven execution control mechanisms, tensor-based workload decomposition methodologies, multi-agent execution synchronization models, and power-aware resource allocation techniques. The invention enables adaptive workload execution, real-time decision-making, and computational energy optimization in distributed computing infrastructures. The disclosed system dynamically allocates execution tasks, optimizes workload migration paths, and continuously adjusts execution schedules based on reinforcement learning-generated execution policies. The system ensures autonomous execution state realignment, minimizes power inefficiencies, and enhances workload scalability through tensor-encoded workload processing architectures, reinforcement learning-based decision matrices, and multi-agent-driven execution coordination mechanisms.

[0002] Existing computational infrastructures face several technical challenges in energy-efficient workload execution, including power inefficiencies in data centres, excessive energy costs in distributed computing environments, execution bottlenecks in real-time systems, workload imbalances in heterogeneous computing architectures, and suboptimal resource utilization leading to inefficient computational scaling. Current execution frameworks lack the ability to dynamically optimize task execution workloads based on real-time power availability constraints, execution demand fluctuations, and reinforcement learning-generated optimization policies. The absence of tensor-based workload

5 decomposition techniques, multi-agent execution synchronization models, and AI-driven power-aware execution scaling frameworks results in computational inefficiencies that significantly impact system-wide execution performance.

[0003] The growing energy consumption in cloud computing environments, the increasing complexity of distributed computing systems, the rising operational
10 costs of large-scale data centres, the environmental impact of high-power computing infrastructure, and the escalating demands for real-time computational processing present significant challenges for existing workload execution systems. Conventional execution models rely on static task scheduling policies, rule-based workload allocation methodologies, and fixed execution dependency mapping
15 techniques that fail to dynamically adjust execution workload intensities based on real-time system conditions, energy efficiency constraints, and task migration optimization policies. These limitations result in excessive energy consumption, prolonged execution latency, execution resource congestion, and reduced computational throughput, particularly in high-performance computing (HPC) and
20 large-scale cloud-based execution environments.

[0004] Conventional task scheduling algorithms suffer from significant limitations, including an inability to dynamically modify execution task priorities based on evolving power constraints, execution latency fluctuations, and workload dependency variations. Existing load-balancing techniques rely on predefined
25 workload allocation heuristics that lack adaptive task execution migration capabilities, resulting in suboptimal resource distribution across heterogeneous execution clusters. Traditional power management systems operate on fixed execution power capping policies that fail to incorporate AI-driven energy forecasting models, reinforcement learning-based execution scheduling techniques, and task workload execution intensity modulation heuristics. Resource allocation
30 methods in conventional systems lack tensor-encoded execution workload processing capabilities, reinforcement learning-driven execution trajectory optimization strategies, and power-aware execution workload redistribution techniques, leading to execution inefficiencies, increased computational energy
35 consumption, and reduced system-wide execution adaptability.

5 [0005] Existing execution workload distribution frameworks fail to incorporate AI-driven workload orchestration methodologies, tensor-based execution adaptation models, and agent-based execution synchronization protocols, resulting in workload execution congestion, execution dependency misalignment, and execution state transition bottlenecks. These limitations prevent existing execution
10 governance systems from autonomously optimizing workload execution sequencing, dynamically modifying execution task allocation parameters, and adaptively regulating computational power usage in real-time distributed computing environments. The absence of reinforcement learning-driven execution optimization techniques further exacerbates execution inefficiencies by preventing
15 execution scheduling policies from dynamically evolving based on execution workload patterns, historical system performance data, and real-time computational energy forecasting models.

[0006] The present invention provides an AI-governed execution workload optimization system that dynamically reconfigures execution workload state
20 pathways, optimizes workload decomposition processes through tensor-based execution segmentation models, synchronizes execution sequences through multi-agent execution negotiation mechanisms, and autonomously modifies execution scheduling policies based on reinforcement learning-enhanced execution decision models. The disclosed system eliminates execution workload inefficiencies by
25 replacing static execution allocation models with tensor-encoded workload fragmentation strategies, reinforcement learning-based execution optimization policies, and agent-driven execution coordination mechanisms. The system enables execution adaptability through AI-driven task classification functions, hierarchical task dependency validation mechanisms, and autonomous workload fragmentation
30 techniques that align execution task scheduling sequences with real-time system power availability states, execution latency thresholds, and power efficiency constraints.

[0007] The disclosed invention provides tensor-based execution optimization capabilities, wherein execution workloads are transformed into multi-dimensional
35 tensor representations that enable fine-grained workload partitioning, hierarchical

5 execution workload ranking, and parallelized task execution optimization. The system further integrates reinforcement learning-based execution scheduling techniques that dynamically refine execution workload migration policies, execution task reallocation functions, and execution prioritization methodologies based on AI-driven execution trajectory optimization models. The invention
10 enhances execution synchronization scalability through multi-agent execution workload negotiation frameworks, ensuring workload execution alignment, execution load distribution balance, and execution state transition optimization in large-scale heterogeneous computing environments.

[0008] The system ensures computational energy efficiency through AI-governed
15 workload execution intensity modulation models, power-aware execution scheduling policies, and energy-efficient execution load rebalancing strategies. The disclosed invention enables workload execution forecasting through AI-enhanced execution performance monitoring functions, execution workload energy consumption modelling algorithms, and reinforcement learning-driven execution
20 trajectory prediction techniques, ensuring execution workload adaptability, scalability, and power efficiency optimization. The disclosed execution workload optimization model autonomously regulates execution workload task migration sequences, execution power state transition protocols, and tensor-encoded execution workload redistribution techniques to dynamically balance
25 computational task allocation across execution nodes while minimizing execution energy consumption overhead.

[0009] Unlike conventional execution workload scheduling models that operate on pre-configured execution task dependency graphs, static workload execution routing models, and non-adaptive execution prioritization frameworks, the
30 disclosed invention incorporates AI-driven execution sequencing algorithms, tensor-based execution segmentation heuristics, and reinforcement learning-driven execution optimization models that dynamically enhance execution efficiency, execution workload scalability, and execution power efficiency. The system continuously modifies execution scheduling policies, execution task sequencing
35 structures, and execution workload balancing heuristics based on reinforcement

5 learning-optimized execution workload decision matrices, ensuring execution scalability, execution workload efficiency, and energy-conscious computational task allocation optimization.

[0010] The disclosed system introduces execution workload synchronization refinement techniques that dynamically adjust execution task dependency mapping
10 models, execution state transition optimization functions, and execution workload distribution adaptation protocols based on reinforcement learning-enhanced execution scheduling algorithms, ensuring execution scalability across large-scale computing environments. The execution workload optimization system enhances computational adaptability through hierarchical execution workload segmentation
15 techniques, AI-driven execution trajectory modification policies, and agent-based execution synchronization methodologies that autonomously regulate execution task scheduling parameters, execution workload balancing coefficients, and execution task prioritization matrices. The system ensures execution state equilibrium through AI-enhanced execution monitoring frameworks, execution
20 workload redundancy mitigation functions, and energy-efficient workload execution adaptation models that dynamically optimize execution task scheduling, execution workload balancing, and execution energy consumption across distributed computing infrastructures.

[0011] The present invention introduces a novel execution workload optimization
25 model that integrates AI-driven execution task sequencing functions, tensor-encoded execution workload fragmentation strategies, reinforcement learning-based execution trajectory refinement techniques, and multi-agent execution synchronization frameworks, ensuring computational workload execution adaptability, execution scheduling scalability, and energy-efficient execution
30 workload optimization. The disclosed execution workload governance system introduces an autonomous execution workload coordination model that dynamically adjusts execution workload scheduling parameters, execution task migration coefficients, and execution workload intensity metrics based on AI-optimized execution scheduling policies, tensor-encoded execution dependency
35 mapping techniques, and reinforcement learning-driven execution workload

5 negotiation models. The system ensures computational execution energy efficiency, execution workload adaptability, and execution scalability through AI-enhanced execution workload alignment methodologies, tensor-based execution workload optimization models, and reinforcement learning-driven execution synchronization techniques that dynamically regulate execution workload state transitions and
10 optimize execution power efficiency across distributed computing infrastructures.

Objectives of the Invention

[0012] The primary objective of the present invention is to provide an AI-governed execution workload optimization system that dynamically modifies computational task execution sequences, optimizes workload distribution pathways, and
15 autonomously adjusts execution scheduling policies based on reinforcement learning-driven decision models, tensor-encoded workload decomposition techniques, and multi-agent execution synchronization mechanisms to enhance computational efficiency, minimize execution latency, and reduce system-wide energy consumption.

20 [0013] Another objective of the invention is to introduce a tensor-based execution management system that enables dynamic workload segmentation, execution granularity control, and real-time workload fragmentation optimization to improve execution scalability and power efficiency.

[0014] Another objective of the invention is to provide a reinforcement learning-
25 driven execution scheduling framework that continuously updates execution policies based on task complexity, power consumption constraints, and real-time execution trajectory optimization, ensuring adaptive execution scheduling and workload prioritization.

[0015] Another objective of the invention is to develop a multi-agent execution
30 synchronization system that ensures distributed workload negotiation, execution state synchronization, and real-time workload reallocation across heterogeneous computing infrastructures.

[0016] Another objective of the invention is to implement an AI-driven power optimization module that dynamically regulates execution energy consumption,

- 5 workload execution intensity, and computational power allocation to enhance system-wide energy efficiency and execution sustainability.
- [0017] Another objective of the invention is to enable grid-aware workload execution management, ensuring real-time adaptation of execution scheduling policies based on power grid fluctuations, execution workload intensity, and power-
- 10 aware execution trajectory adjustments.
- [0018] Another objective of the invention is to introduce a fault-tolerant execution workload system that autonomously detects execution failures, reallocates failed tasks, and dynamically restores execution workload stability through distributed failure recovery mechanisms.
- 15 [0019] Another objective of the invention is to develop a self-optimizing execution workload management framework that dynamically adjusts execution workload sequencing, task prioritization, and resource allocation based on AI-driven execution monitoring, workload dependency modelling, and tensor-based execution optimization methodologies.
- 20 [0020] Another objective of the invention is to provide a workload execution forecasting system that utilizes predictive modelling to anticipate execution demand variations, optimize workload sequencing, and refine execution workload dependency synchronization in distributed computing environments.
- [0021] Another objective of the invention is to establish a distributed execution
- 25 workload governance system that enables real-time execution workload transition optimization, adaptive execution power modelling, and reinforcement learning-based workload execution refinement to ensure efficient workload execution across distributed computing clusters.

Summary of the Invention

- 30 [0022] The present invention relates to a system and method for energy-efficient computing, wherein computational task execution, workload distribution, and execution synchronization are governed by an artificial intelligence-driven workload optimization framework. The system implements tensor-based workload decomposition, reinforcement learning-based scheduling, multi-agent execution
- 35 coordination, and power-aware execution scaling mechanisms to facilitate adaptive

5 task execution, workload realignment, and resource efficiency in distributed computing environments.

[0023] The system comprises an artificial intelligence-controlled workload management module configured to classify execution workloads based on task complexity, execution dependency structures, and computational power allocation

10 constraints. The system further includes a tensor-based execution control module, wherein workload processing operations are executed through multi-dimensional tensor decomposition methodologies, hierarchical workload segmentation procedures, and execution resource redistribution strategies, enabling optimized workload execution sequencing, execution resource allocation, and execution task

15 migration.

[0024] The system integrates a reinforcement learning-based execution scheduling module, wherein execution workload transition states are dynamically optimized based on workload dependency constraints, execution task sequencing heuristics, and computational power availability assessments. The system further includes a

20 multi-agent execution synchronization module, wherein execution task coordination, real-time workload distribution, and workload migration are facilitated through distributed execution decision models, execution workload synchronization frameworks, and agent-based execution workload adaptation mechanisms.

25 [0025] The system further comprises an artificial intelligence-based power optimization module, wherein execution task energy consumption, workload execution intensity modulation, and execution power allocation adjustments are dynamically governed based on real-time computational energy constraints, execution workload prioritization parameters, and workload execution trajectory

30 optimization functions. The system includes a grid-aware execution workload management module, wherein execution workload scheduling parameters, workload execution state adjustments, and execution workload prioritization sequences are dynamically reconfigured in response to power grid stability conditions, computational energy allocation constraints, and real-time execution

35 workload distribution requirements.

5 [0026] The system further comprises a failure-resilient execution workload control
framework, wherein execution failure detection, workload execution fault recovery,
and execution workload realignment procedures are dynamically executed through
failure detection models, execution task reassignment methodologies, and
hierarchical execution workload synchronization procedures. The system further
10 includes an execution workload governance framework, wherein workload
execution sequencing, execution state transition mapping, and execution scheduling
modifications are dynamically adjusted based on workload execution dependency
evaluation models, execution workload intensity calibration techniques, and
execution workload synchronization refinement strategies.

15 [0027] The system provides execution workload forecasting capabilities, wherein
execution workload demand variations, execution workload transition adjustments,
and execution workload scheduling modifications are dynamically predicted and
optimized through execution workload forecasting models, execution task
sequencing algorithms, and computational power allocation functions. The system
20 includes a distributed execution workload coordination framework, wherein
execution workload transition alignment, execution workload synchronization
adjustments, and execution workload execution energy modelling procedures are
dynamically refined based on execution scheduling optimization policies,
execution workload dependency mapping methodologies, and execution workload
25 execution trajectory evaluation techniques.

Detailed Description of Figures

[0028] FIG. 1 illustrates a system overview and AI-governed execution framework
(100) configured to manage computational workload execution within a distributed
30 execution environment. The system (100) comprises a system initialization module
(101) configured to define execution parameters, establish execution prioritization,
and load baseline execution policy models. An execution coordination layer (102)
is communicatively coupled with an AI-based workload optimization engine (103),
which receives execution state updates from a tensorized workload assignment
35 module (104). The tensorized workload assignment module (104) adjusts

5 computational workload allocations based on task prioritization computed by a reinforcement learning execution scheduler (105). The system (100) includes a multi-agent workload execution governance module (106) comprising independent execution nodes (107) connected via a real-time execution synchronization protocol (108). A workload fragmentation processor (109) analyzes task complexity,
10 execution timing requirements, and power efficiency metrics. A hierarchical task allocation framework (110) applies an execution density scoring algorithm (111), enabling an AI-controlled decision matrix (112) to determine execution assignments across heterogeneous processing nodes. The system (100) further comprises an execution error handling module (113) configured to monitor
15 execution failures, workload discrepancies, and processing node synchronization errors. An error detection engine (114) classifies execution failures based on severity levels and routes error resolution commands to an AI-based fault mitigation processor (115), which applies workload correction policies based on failure analysis computed by an execution failure classification module (116). A real-time
20 execution communication protocol (117) exchanges execution workloads, error handling updates, and workload migration requests between execution nodes via a multi-channel workload messaging bus (118).

[0029] FIG. 2 depicts a tensor-based workload decomposition and execution
25 assignment system (200) configured to process computational workloads through an AI-based tensor encoding framework. The system (200) comprises an execution state initialization module (201) configured to define workload segmentation parameters, execution dependency matrices, and workload complexity assignment models prior to execution dispatch. An AI-based workload failure recovery module
30 (202) adjusts execution pathways in response to tensor misallocations detected by a tensor error identification engine (203). Execution anomalies are classified through a fault categorization module (204) applying tensor consistency verification heuristics to determine whether execution tensors require reprocessing within a tensor recomposition controller (205). The system (200) integrates a
35 workload migration adaptation engine (206), wherein execution workloads

5 requiring path adjustments due to power fluctuations, timing misalignment, or node failures are reassigned across tensor execution clusters based on load balancing computations performed within an execution efficiency optimization module (207). An execution tensor synchronization protocol (208) exchanges tensorized execution workload messages, reallocation signals, and computational feedback
10 packets within a multi-node workload execution bus (209).

[0030] FIG. 3 illustrates a reinforcement learning model for execution path optimization (300) configured to adjust workload execution sequences based on a reinforcement learning execution prioritization framework (301). A system
15 bootstrapping module (302) initializes state-action-reward training matrices and synchronizes workload execution datasets prior to scheduling. An execution policy adaptation module (303) modifies execution scheduling heuristics based on outcome evaluations performed within a reinforcement learning execution state analysis module (304). Execution workloads requiring scheduling adjustments due
20 to model performance deviations are modified within an execution path optimization controller (305). The system (300) comprises an AI-based execution failure correction module (306), wherein detected inconsistencies are restructured through a hierarchical execution rollback engine (307), enabling failed execution states to undergo realignment within an execution trajectory recalibration processor
25 (308). A reinforcement learning synchronization network (310) exchanges reinforcement learning model updates, priority reassignment notifications, and trajectory reoptimization commands via an execution workload communication protocol (309).

30 [0031] FIG. 4 illustrates a fault-tolerant execution and recovery mechanism (400) comprising an AI-based execution fault tolerance module (401) configured to detect, isolate, and recover from computational execution anomalies. A real-time failure monitoring engine (402) analyzes workload execution integrity and applies failure recovery policies through an execution rollback processor (403). Workload
35 reallocation strategies are executed via an execution checkpointing ledger (404)

5 configured to periodically store execution snapshots. A failure prediction module (405) anticipates execution failures by analysing historical performance and applying a probabilistic workload error classification engine (406) to determine failure risk coefficients. Pre-emptive workload redistribution is enabled via an execution stability optimization module (407). An execution state synchronization
10 recovery controller (408) ensures rollback operations maintain consistency across dependent workloads. A hierarchical execution resilience model (409) classifies workloads based on priority levels and applies a fault-tolerance matrix (410). An AI-based resource reallocation unit (411) reassigns workloads to alternative nodes.

15 [0032] FIG. 5 depicts an AI-governed execution synchronization and node communication system (500) comprising an execution synchronization module (501) configured to update execution workloads across distributed processing nodes via a real-time execution state messaging engine (502). Execution state updates are transmitted using a workload synchronization protocol (503). A latency-aware
20 execution synchronization controller (504) prioritizes state updates based on workload urgency and manages workload dependencies through an execution coordination module (505). A multi-channel execution coordination layer (506) distributes parallel execution messaging across nodes through a reinforcement learning-based execution consensus module (507). A fault-tolerant node
25 communication processor (508) reroutes synchronization messages across alternative nodes during communication failure. An execution clock synchronization controller (509) maintains timing precision across nodes.

[0033] FIG. 6 illustrates an adaptive workload execution and load balancing
30 framework (600) comprising an AI-based workload execution balancing module (601) configured to distribute workloads based on execution pressure analytics computed via a workload execution pressure balancer (602). Node utilization is monitored, and workloads are redistributed via an execution load redistribution policy optimizer (603). A queue delay prediction engine (604) analyzes execution
35 queue congestion and adjusts workload assignments using an execution timing

5 optimizer (605). A task dependency awareness module (606) prevents dependency fragmentation by retaining interdependent workloads within a single cluster. An energy-aware execution redistribution controller (607) reallocates workloads based on power consumption analytics.

10 [0034] FIG. 7 illustrates a self-learning execution refinement and AI adaptation system (700) comprising an AI-based execution self-learning module (701) configured to modify workload execution strategies based on historical pattern analysis computed via an execution pattern learning engine (702). Execution inefficiencies are detected, and optimization heuristics are applied through a self-
15 adaptive execution refinement processor (703). An execution anomaly auto-correction module (704) realigns workloads when deviations from predefined efficiency thresholds are detected. Workload execution policies are updated via a real-time execution optimization feedback loop (705). A workload execution stability forecasting system (706) predicts stability trends and adjusts scheduling.

20

[0035] FIG. 8 illustrates an AI-governed grid power synchronization and execution planning system (800) comprising a grid-aware execution synchronization engine (801) configured to adjust workload execution based on grid power availability analysis computed via a grid energy forecasting module (802). Energy demand
25 fluctuations are estimated, and scheduling strategies are modified using a grid stability-aware execution scheduling engine (803). A load-balanced execution reallocation processor (804) redistributes workloads based on power stability trends. A renewable energy optimization engine (805) aligns workload prioritization with renewable energy availability metrics. An execution energy
30 consumption prediction model (806) forecasts energy demands and adjusts workloads.

Detailed Description of Invention

[0036] FIG. 1 illustrates a system (100) for dynamic computational workload execution comprising a computational task allocation module (101) operatively
35 coupled to a self-organizing execution control subsystem (102) via a feedback-

5 governed execution adaptation layer (107). The system integrates a tensor-encoded workload decomposition engine (103) (FIG. 2) configured to restructure workloads into multi-dimensional execution graphs using singular value decomposition (SVD) and low-rank approximation. A neural reinforcement decision matrix (104) (FIG. 3) implements a reward function $R(s, a) = \alpha(E_{saved}) + \beta(P_{reduced}) - \gamma(T_{delay})$,
 10 where E_{saved} , $P_{reduced}$, and T_{delay} denote energy efficiency gains, power savings, and timing penalties. An AI-regulated energy allocation framework (105) (FIG. 8) dynamically modulates workload assignments through a probabilistic energy distribution function refined via real-time feedback from a tensor error identification engine (203) (FIG. 2). A distributed execution optimization module
 15 (106) (FIG. 6) redistributes workloads across heterogeneous processing nodes (107) based on real-time power constraints and thermal stability thresholds (409) (FIG. 4).

[0037] The adaptive tensor-encoded power optimization function (FIG. 2) represents computational tasks as multi-dimensional tensors using a Haar wavelet-based decomposition framework. The tensor processing module (103) applies
 20 energy-adaptive transformation matrices to adjust execution complexity coefficients, governed by a gradient descent protocol minimizing energy loss $L(t) = \sum_{i=1}^N P(T_i) \cdot E(T_i) - \lambda \cdot \nabla_T W_t$, where $P(T_i)$ denotes power draw and $E(T_i)$ is execution duration. A federated tensor allocation function (207) (FIG. 2)
 25 evaluates real-time power consumption thresholds using a convolutional neural network trained on historical execution failure patterns. Progressive tensor recomposition operations (205) merge decomposed segments into optimized execution pathways aligned with energy-efficiency benchmarks. The framework dynamically adjusts coefficients using transformation matrices governed by
 30 gradient descent, ensuring alignment with real-time grid stability metrics (803) (FIG. 8).

[0038] A multi-agent reinforcement learning negotiation model (104) (FIG. 3) deploys independent AI decision agents across nodes (107) to optimize workload distribution, power adaptation, and risk mitigation. Agents engage in iterative
 35 negotiation sequences using a decentralized consensus mechanism (507) (FIG. 5)

5 to synchronize policies across a hierarchical state-action-value matrix (304). Fault categorization (204) (FIG. 2) applies tensor consistency heuristics to detect misallocations, rerouting anomalies to a tensor recomposition controller (205) (FIG. 2) for reprocessing. The policy is trained via federated reinforcement learning (310) (FIG. 3), ensuring decentralized synchronization of decision states. Execution
10 anomalies are classified using a probabilistic workload error classification engine (406) (FIG. 4).

[0039] The AI-controlled power resource arbitration module (105) (FIG. 8) modulates workload assignments via grid stability-aware scheduling (803) and predictive trajectory modelling. A load-sensitive execution balancing function
15 (602) (FIG. 6) shifts workloads in response to grid fluctuations, mitigating latency through adaptive synchronization protocols (503) (FIG. 5). The probabilistic energy distribution function is refined via real-time feedback from an execution efficiency optimization module (207) (FIG. 2). Pre-emptive workload redistribution (407) (FIG. 4) is triggered by risk coefficients computed via a probabilistic error
20 classification engine (406). The arbitration module (105) integrates a node-level execution hierarchy refinement layer (607) (FIG. 6) to dynamically assign power coefficients based on historical trends.

[0040] The tensor-decomposed workload execution framework (103) (FIG. 2) segments tasks via a hierarchical tensor processing module (201) using SVD and
25 energy-scaled decomposition. High-complexity workloads are partitioned into tensor segments proportional to node power thresholds, with decomposition parameters defined by an execution state initialization module (201) (FIG. 2). The federated tensor allocation function (207) aligns matrices with energy-efficiency benchmarks using reinforcement learning-derived load balancing (603) (FIG. 6).
30 Progressive recomposition operations (205) merge decomposed segments into optimized pathways via the AI-controlled decision matrix (112) (FIG. 1). The framework dynamically adjusts coefficients using transformation matrices governed by gradient descent, ensuring continuous alignment with grid stability metrics (803) (FIG. 8).

5 [0041] A self-optimizing reinforcement learning scheduler (305) (FIG. 3) adjusts workload states via a hierarchical state-action-value matrix (304) trained using double Q-learning. Federated policy training (303) synchronizes heuristics across nodes via Lamport clock-based protocols (509) (FIG. 5). The scheduler computes optimal policies via a probabilistic energy-state trajectory model (304), applying

10 Thompson sampling for task prioritization. Execution path realignment is performed using a decentralized synchronization protocol (208) (FIG. 2), with non-convergent trajectories corrected via a latency compensation mechanism (509) (FIG. 5). The scheduler integrates a federated reinforcement policy aggregation framework (310) (FIG. 3) to ensure global convergence.

15 [0042] The AI-governed thermal management framework (401) (FIG. 4) reallocates workloads using thermographic mapping data from a temperature anomaly detection engine (402). Nodes exceeding thermal stability thresholds (409) are assigned low-power workloads via a redistribution module (407), with heat dissipation protocols modulating execution density using thermal saturation

20 analytics. Real-time thermographic data is analysed via a recurrent neural network (702) (FIG. 7) to pre-emptively mitigate throttling risks. Workloads are prioritized based on thermal gradients computed by an AI-driven fault mitigation processor (115) (FIG. 1). The framework dynamically adjusts workload execution density (110) (FIG. 1) to prevent localized hotspots using a hierarchical execution resilience

25 model (409).

[0043] A federated AI execution synchronization module (310) (FIG. 3) propagates policies via a federated learning consensus model (507) (FIG. 5), exchanging execution state vectors between nodes (107) (FIG. 1) through a multi-channel workload messaging bus (118). A distributed latency compensation mechanism

30 (509) (FIG. 5) corrects non-convergent trajectories via iterative synchronization, ensuring alignment with global power minimization objectives. The module enforces grid stability compliance using a renewable energy optimization engine (805) (FIG. 8). Execution state vectors are validated via an execution state synchronization recovery controller (408) (FIG. 4) to prevent desynchronization.

5 [0044] The AI-controlled execution prioritization framework (112) (FIG. 1) realigns workloads via a multi-tier scaling function, with priority assignments validated by an execution state validation module (701) (FIG. 7). A demand forecasting engine (604) (FIG. 6) analyzes historical data and load distribution factors using long short-term memory (LSTM) networks. Workload queues are

10 dynamically reordered based on probabilistic metrics from a reinforcement-driven engine (303) (FIG. 3). Real-time recalibration (605) aligns tasks with demand constraints via an execution trajectory refinement model (705) (FIG. 7). The framework integrates a workload adaptive real-time execution recalibration module (605) (FIG. 6) to modulate task fragmentation states.

15 [0045] The multi-layer power adaptation framework (601) (FIG. 6) reallocates tasks across electronic, neuromorphic, and photonic nodes (508) (FIG. 5) using a state transition protocol. A hierarchy refinement layer (607) assigns power coefficients based on historical trends analysed by an execution pattern learning engine (702) (FIG. 7). Workloads with high dynamic power consumption gradients undergo

20 reclassification via an execution stability scoring matrix (603). The framework ensures alignment with energy efficiency objectives using the AI-regulated energy allocation module (105) (FIG. 8). Node-level execution hierarchy parameters are dynamically updated via a reinforcement learning-driven policy optimizer (603) (FIG. 6).

25 [0046] FIG. 5 illustrates a photonic-electronic execution synchronization framework (500) comprising a lightwave processing module (502) configured to modulate workloads assigned to photonic nodes (508) using a probabilistic wavelength alignment function (503). The framework integrates a multi-stage wavelength multiplexing engine (505) to distribute execution states across multi-

30 wavelength optical nodes (508) via a multi-node workload execution bus (209) (FIG. 2). Execution throughput is optimized through adaptive synchronization of electronic state-driven controllers (506) (FIG. 5) with optically-modulated nodes (508). A fault-tolerant node communication processor (508) reroutes synchronization messages during failures using a Paxos consensus protocol. Timing

35 precision is maintained via an execution clock synchronization controller (509)

5 (FIG. 5) that calibrates timestamps using reinforcement learning-derived forecasts (304) (FIG. 3).

[0047] FIG. 1 and FIG. 6 depict a hierarchical task fragmentation module (109, 606) that partitions workloads via a self-optimizing algorithm (703) (FIG. 7) using spectral clustering. Correlated complexity profiles are aggregated into execution

10 clusters (606) via k-means clustering, governed by a reinforcement-driven workload clustering mechanism (303) (FIG. 3). The trajectory optimization module (705) (FIG. 7) refines segmentation heuristics using gradient descent, aligning fragmentation states with power benchmarks via the federated tensor allocation function (207) (FIG. 2). Dependency fragmentation is prevented through a task

15 dependency awareness engine (606) (FIG. 6) that retains interdependent workloads within clusters.

[0048] The real-time execution cost analysis module (FIG. 7) monitors overhead parameters (702) using autoregressive integrated moving average (ARIMA) models. An efficiency coefficient computation engine (703) generates

20 minimization heuristics via convex optimization, dynamically refining execution trajectories through an AI-driven feedback loop (705). Predictive execution cost forecasting (706) modulates workloads using support vector machine (SVM)-based projections. Cost directives are propagated across the computational network via a multi-channel workload messaging bus (118) (FIG. 1). The module ensures

25 alignment with energy-efficiency benchmarks using the execution state validation framework (701) (FIG. 7).

[0049] The tensorized power load management module (207) (FIG. 2) balances loads via an AI-driven adaptation function (601) (FIG. 6), evaluating congestion risks through Monte Carlo simulations. Workloads are redistributed via an

30 execution load redistribution policy optimizer (603) using reinforcement learning-derived coefficients. Latency prevention is enforced through an execution timing optimizer (605) (FIG. 6) that applies Dijkstra's algorithm for path selection. Load balancing coefficients are dynamically updated via a policy optimizer (603) (FIG. 6) using federated learning consensus (507) (FIG. 5).

5 [0050] FIG. 2 illustrates an AI-driven workload migration framework (206) that evaluates multi-variable factors (power, latency, thermal states) using multi-variable regression. Workloads are reassigned to optimal nodes (107) (FIG. 1) via a reprioritization protocol (505) (FIG. 5) governed by Hungarian algorithm-based matching. The execution reclassification function (408) (FIG. 4) recalculates

10 priorities using stability scoring matrices (603) (FIG. 6). Migration commands are propagated via a reinforcement learning synchronization network (310) (FIG. 3), ensuring compliance with power benchmarks through the validation module (701) (FIG. 7).

[0051] The adaptive multi-agent energy arbitration framework (408) (FIG. 4)

15 negotiates power allocation via decentralized consensus (507) (FIG. 5) using Nash equilibrium principles. Agents compute optimal execution locations through iterative negotiation sequences (304) (FIG. 3), resolving energy discrepancies via real-time synchronization (208) (FIG. 2). The probabilistic equilibrium function (410) measures deviations using the execution state analysis module (304) (FIG. 3).

20 Arbitration policies are refined via federated learning (310) (FIG. 3), ensuring alignment with global power minimization objectives (803) (FIG. 8).

[0052] The hybrid reinforcement learning synchronization module (310) (FIG. 3) aligns execution trajectories via federated optimization (803) (FIG. 8). Redundancies and latency penalties are adjusted through a hierarchical workload

25 redistribution function (506) (FIG. 5). A trajectory refinement engine (705) (FIG. 7) propagates heuristics across controllers using gradient averaging, harmonizing states via a consensus module (507) (FIG. 5). Non-convergent execution trajectories undergo correction through a transition correction module (509) (FIG. 5) applying Byzantine fault tolerance.

30 [0053] FIG. 8 depicts a grid-integrated execution planning framework (801) that coordinates workload scheduling with grid fluctuations via an AI-based grid energy forecasting module (802). Workload densities are recalibrated using a fragmentation engine (805) applying mixed-integer linear programming (MILP). The scheduling engine (803) prioritizes renewable energy metrics (805) through

35 SVM-based forecasts. The prediction model (806) dynamically adjusts workloads

5 using autoregressive forecasting, ensuring alignment with power stability trends (804).

[0054] The self-optimizing segmentation framework (109) (FIG. 1) partitions high-complexity tasks into scalable fragments (207) (FIG. 2) using k-nearest neighbors (k-NN) clustering. A synchronization function (208) (FIG. 2) reassigns fragments

10 via optimization heuristics derived from the reinforcement learning optimizer (303) (FIG. 3). Dependency awareness is enforced through an engine (606) (FIG. 6) that prevents fragmentation using Petri net-based scheduling. Workload states are dynamically modulated via the trajectory refinement model (705) (FIG. 7).

[0055] The workflow automation framework (506) (FIG. 5) manages dependencies

15 via an AI governance model (505) implementing Lamport clock-based synchronization (509). A transition controller (508) modulates handoffs between electronic and photonic nodes (508) using greedy algorithms. Parallel processing assignments are optimized through an execution coordination layer (506) (FIG. 5). Policy recommendations are generated via the reinforcement learning module (303)

20 (FIG. 3), with real-time adjustments validated by the state validation module (701) (FIG. 7).

[0056] The multi-variable power adaptation framework (601) (FIG. 6) realigns computational tasks using stability scoring matrices (603) derived from Monte Carlo simulations. A cost optimization module (605) evaluates energy efficiency

25 coefficients via dynamic programming, integrating execution energy consumption prediction models (806) (FIG. 8). Resource utilization is validated through a checkpointing ledger (404) (FIG. 4), ensuring alignment with federated learning protocols (310) (FIG. 3). The framework applies Petri net-based scheduling to manage workload dependencies and parallel processing assignments. Execution

30 states are dynamically recalibrated using stability thresholds computed by the execution efficiency engine (702) (FIG. 7).

[0057] The neural execution prediction framework (702) (FIG. 7) employs recurrent neural networks (RNNs) to forecast execution trajectories, with deviations corrected via Hungarian algorithm-based reallocation (704). Predictive analytics

35 are refined through an autoregressive integrated moving average (ARIMA) model

5 (706), pre-emptively adjusting workloads using gradient descent. Execution state projections are validated through a checkpointing ledger (404) (FIG. 4) storing historical performance snapshots. The module ensures compliance with power benchmarks via the execution state validation engine (701) (FIG. 7).

[0058] The execution risk mitigation framework (401) (FIG. 4) monitors fault
 10 tolerance parameters (402) using Bayesian networks to compute failure probabilities. A resilience function (407) allocates redundancy via erasure coding, ensuring real-time computational stability. Failed execution states are recovered using snapshots from the checkpointing ledger (404), validated through the execution state analysis module (304) (FIG. 3). Probabilistic forecasts of execution
 15 deviations are generated via Markov chain Monte Carlo (MCMC) sampling. The framework enforces compliance with thermal stability thresholds (409) (FIG. 4).

[0059] The execution path selection framework (112) (FIG. 1) computes scoring metrics via probabilistic evaluation (304) using Kalman filters. Path optimization is performed via Dijkstra's algorithm, with congestion prevention enforced by the
 20 load balancer (602) (FIG. 6). Resource adaptation protocols (603) align states with availability trends using federated learning consensus (507) (FIG. 5). Execution trajectories are validated via the state synchronization recovery controller (408) (FIG. 4). The framework ensures alignment with grid stability metrics (803) (FIG. 8).

25 [0060] The reinforcement learning workflow optimizer (303) (FIG. 3) modulates scheduling sequences via policy refinement (310) using federated averaging. A priority reallocation function (605) (FIG. 6) reorders queues via greedy algorithms, harmonizing states through a consensus module (507) (FIG. 5). Latency minimization is enforced by the synchronization controller (504) (FIG. 5), applying
 30 Paxos protocols to resolve discrepancies. Execution policies are dynamically propagated across nodes (107) (FIG. 1) via the multi-channel messaging bus (118).

[0061] The distributed AI synchronization framework (310) (FIG. 3) propagates heuristics via federated averaging, resolving discrepancies using Paxos consensus. Non-convergent states undergo correction through a transition correction module
 35 (509) (FIG. 5) implementing Byzantine fault tolerance. Synchronization accuracy

5 is validated via the execution clock controller (509) (FIG. 5). The framework ensures equilibrium through real-time analysis (304) (FIG. 3) of execution trajectories.

[0062] The real-time load balancing framework (602) (FIG. 6) prevents saturation via spectral clustering-based partitioning (109) (FIG. 1). Workloads are

10 redistributed using reinforcement learning-derived coefficients (603), validated through the state validation module (701) (FIG. 7). Density optimization is enforced via the federated tensor allocation function (207) (FIG. 2). Self-healing operations (407) (FIG. 4) rectify deviations using erasure coding.

[0063] The self-learning policy refinement framework (703) (FIG. 7) calibrates

15 decisions via Q-learning, realigning policies using Nash equilibrium principles. Policy transitions are computed via Markov decision processes (MDPs), synchronized through the decentralized protocol (208) (FIG. 2). Calibration functions (706) dynamically adjust scoring matrices (603) using gradient descent. The framework ensures equilibrium via Lamport clock-based synchronization

20 (509) (FIG. 5).

[0064] The execution state validation module (701) (FIG. 7) verifies compliance via anomaly detection heuristics (704) using Bayesian networks. Self-healing operations (407) (FIG. 4) rectify deviations through Byzantine fault tolerance. Probabilistic forecasts of execution discrepancies are generated via autoregressive

25 forecasting (706). The module ensures continuity through snapshots stored in the checkpointing ledger (404) (FIG. 4).

[0065] The policy enforcement framework (701) (FIG. 7) validates compliance via trajectory verification (705), resolving anomalies through iterative calibration (704). Adaptive decision-making (505) (FIG. 5) applies Thompson sampling to

30 optimize workload assignments. Execution policies are dynamically propagated via the federated synchronization protocol (310) (FIG. 3). The framework ensures adherence to energy benchmarks through the validation engine (701).

[0066] FIG. 7 illustrates a real-time execution policy enforcement framework (701) configured to validate compliance via AI-powered trajectory verification (705).

35 Anomalies are resolved through iterative calibration sequences (704) using

5 Thompson sampling and federated learning consensus (507) (FIG. 5). Adaptive
decision-making (505) (FIG. 5) optimizes workload assignments via autoregressive
integrated moving average (ARIMA) models (706). Execution policies are
dynamically propagated across nodes (107) (FIG. 1) through a multi-channel
workload messaging bus (118), ensuring adherence to energy benchmarks via the
10 validation engine (701). The framework integrates a reinforcement learning-based
anomaly resolution module (704) that applies Byzantine fault tolerance to rectify
deviations. Compliance with thermal stability thresholds (409) (FIG. 4) is enforced
through continuous monitoring by the temperature-aware redistribution module
(407).

15 [0067] FIG. 8 depicts a comprehensive execution model integration framework
(801) automating lifecycle management via AI-driven monitoring (802) and MILP-
based scheduling (803). The self-adaptive protocol (805) refines heuristics using
federated learning (310) (FIG. 3), ensuring workload execution alignment with
renewable energy availability metrics (805). Execution path selection, task
20 sequencing, and resource utilization strategies are autonomously determined
through Markov decision processes (MDPs) (705) (FIG. 7). The framework ensures
long-term equilibrium via the execution clock synchronization controller (509)
(FIG. 5), maintaining compliance with power stability trends (804). Workload
execution parameters are stored in the checkpointing ledger (404) (FIG. 4) for
25 historical validation. The module enforces system-wide energy optimization
through continuous feedback from the execution efficiency engine (702) (FIG. 7).

Embodiments

[0068] In one embodiment, an AI-driven execution system for energy-efficient data
30 centres is disclosed, wherein the system dynamically reallocates computational
tasks based on real-time energy availability and thermal efficiency metrics. A
reinforcement learning-based workload redistribution module ensures that tasks are
migrated to low-energy processing nodes, thereby optimizing power consumption
without compromising execution performance.

5 [0069] Another embodiment pertains to distributed cloud computing infrastructures, where an autonomous workload arbitration model dynamically negotiates task distribution across geographically dispersed execution clusters. The system continuously evaluates power efficiency constraints, bandwidth utilization, and execution latency, ensuring optimal AI-driven workload scheduling across the distributed network.

10 [0070] In yet another embodiment, a blockchain-based execution verification framework is provided, wherein cryptographic workloads are dynamically scheduled based on AI-driven energy-aware consensus protocols. The system ensures that cryptographic operations, including hash computations and smart contract verifications, are assigned to nodes exhibiting the lowest computational overhead, thereby minimizing energy redundancy and optimizing ledger synchronization.

15 [0071] One embodiment describes an AI-powered workload optimization system for next-generation wireless networks, wherein execution scheduling dynamically adapts based on real-time network traffic analysis and spectrum efficiency constraints. The system ensures that latency-sensitive AI execution models, such as predictive analytics and signal optimization tasks, are prioritized within bandwidth-constrained network environments.

20 [0072] In an alternative embodiment, an execution sequencing model for high-performance computing (HPC) clusters is provided, wherein tensor-based workload fragmentation ensures computational tasks are dynamically restructured and allocated based on processor availability, power consumption trends, and cooling efficiency. The system implements tensor decomposition techniques to enhance task scalability across multi-node supercomputing environments.

25 [0073] Disclosed herein is an AI-powered financial risk modelling execution framework, wherein workload scheduling dynamically adapts based on real-time market conditions, transaction volume fluctuations, and risk assessment priorities. The system autonomously assigns computational resources to mission-critical AI-driven risk simulations, ensuring adaptive execution workload prioritization under constrained processing conditions.

30

35

5 [0074] According to another embodiment, an autonomous vehicle AI workload execution system is presented, wherein execution models dynamically transition between on board and cloud-based processing environments. AI-driven perception models continuously analyse battery constraints, real-time traffic patterns, and obstacle recognition data, ensuring workload redistribution aligns with energy efficiency requirements for autonomous navigation.

10 [0075] In yet another aspect, an industrial automation workload scheduling system is disclosed, wherein hierarchical AI execution workflows dynamically modify robotic process automation sequences. The system continuously monitors sensor feedback and production constraints, ensuring AI-driven process optimization models adapt to real-time assembly line variations without exceeding energy constraints.

15 [0076] In another embodiment, an AI-governed cybersecurity execution model is provided, wherein anomaly detection algorithms dynamically schedule workload execution across distributed network security clusters. AI-driven workload reallocation ensures that threat detection, forensic analysis, and intrusion prevention tasks are executed with minimal computational overhead while maintaining optimal security posture.

20 [0077] Another disclosed embodiment relates to autonomous drone swarm coordination, wherein execution workloads are dynamically distributed based on mission objectives, real-time environmental conditions, and UAV battery efficiency models. AI-powered trajectory prediction ensures that flight path computation tasks are adaptively reassigned between airborne and ground-based processing units, optimizing execution sequencing.

25 [0078] A further embodiment pertains to precision agriculture AI models, where workload execution dynamically aligns with weather forecasts, soil moisture conditions, and crop health monitoring data. AI-driven execution synchronization ensures that predictive irrigation, pesticide application, and harvesting task models adapt in real time to optimize agricultural efficiency.

30 [0079] In an additional embodiment, an AI-driven satellite-based computational framework is disclosed, wherein reinforcement learning algorithms continuously

35

5 adjust execution workload distribution for Earth observation, remote sensing, and
geospatial analysis. AI-powered execution task balancing ensures that satellite
computational workloads remain optimized across multi-orbit deployment
networks.

[0080] In another aspect, an AI-enhanced video analytics execution model is
10 presented, wherein execution workloads dynamically adjust based on real-time
video processing demand and computational load balancing. The system
autonomously partitions video analytics workloads into parallel execution threads,
optimizing motion tracking, facial recognition, and behavioural analysis tasks.

[0081] One disclosed embodiment describes an AI-powered biomedical imaging
15 execution framework, wherein diagnostic inference tasks are dynamically assigned
across distributed AI-driven radiology processing clusters. The system
continuously evaluates execution demand variations to optimize workload
segmentation for MRI, CT scan analysis, and real-time medical anomaly detection
tasks.

20 [0082] An alternative embodiment pertains to hybrid quantum-AI execution
scheduling, wherein AI-driven tensor execution models dynamically allocate
computationally intensive workloads between quantum and classical computing
clusters. The system autonomously transitions execution workflows to minimize
quantum DE coherence effects, ensuring optimal quantum-to-classical execution
25 synchronization for AI-powered quantum simulation workloads.

[0083] Another embodiment describes an AI-powered digital twin execution
synchronization model, wherein execution tasks dynamically adjust based on
predictive maintenance simulations, urban planning analytics, and industrial
process modelling. AI-driven execution transitions ensure that simulated workload
30 models remain dynamically aligned with real-world execution constraints.

[0084] In yet another embodiment, an AI-enhanced sports analytics execution
system is provided, wherein workload sequencing dynamically modifies athlete
motion tracking models, biometric performance analytics, and predictive injury
assessment algorithms. The execution workload synchronization framework
35 ensures real-time adaptation to live game event dynamics.

5 [0085] In an additional embodiment, an AI-driven natural language processing
execution scheduler is disclosed, wherein real-time NLP execution workloads
dynamically adapt to speech pattern recognition models, multilingual translation
engines, and AI-driven semantic understanding frameworks. AI-based execution
state modification ensures workload synchronization across voice recognition AI
10 clusters.

[0086] One embodiment pertains to an AI-powered air traffic management
execution model, wherein real-time execution task prioritization dynamically
modifies flight scheduling, air traffic rerouting, and congestion forecasting tasks.
AI-driven execution synchronization ensures workload balancing across distributed
15 aviation control systems.

[0087] Another embodiment discloses an AI-driven pharmaceutical drug discovery
execution model, wherein AI-powered chemical compound simulations
dynamically modify execution tasks to optimize drug interaction analysis,
molecular property prediction, and drug efficacy evaluation. Execution workload
20 segmentation ensures computational resources are allocated efficiently across AI-
driven simulation clusters.

25

30

35

5 **5. Claims**

I Claim:

1. A system for AI-driven energy-efficient computing and tensor-based workload optimization, the system comprising:

- **An AI-governed workload management module, wherein:**

- 10 ○ **A neural network-based execution classifier** analyzes **workload characteristics** based on processing intensity, memory access patterns, and I/O bandwidth constraints.
- **A reinforcement learning-driven task scheduler** using **Advantage Actor-Critic (A2C) models** dynamically distributes
- 15 workloads across **heterogeneous computing nodes (CPU, GPU, TPU, and FPGA clusters)**.
- **A workload execution predictor**, trained on **long short-term memory (LSTM) models**, estimates execution bottlenecks and optimizes scheduling parameters.
- 20 ○ **A tensor representation engine**, encoding workload parameters into **high-dimensional embeddings** for efficient execution mapping.

2. The system as claimed in claim 1, further comprising a tensor-based execution control module, implementing: Multi-level tensor decomposition techniques, wherein:

- **Canonical Polyadic Decomposition (CPD)** is used for fine-grained workload partitioning, modeled as:

$$X \approx \sum_{r=1}^R \lambda_r \otimes a_r \otimes b_r \otimes c_r$$

30 where X is the workload tensor, and λ_r represents adaptive workload segmentation factors.

- **Tucker Decomposition (TD)** optimizes memory footprint via:

$$X \approx G \times_1 U_1 \times_2 U_2 \times_3 U_3$$

35 with dynamic adjustment of factor matrices U_i based on real-time execution constraints.

- 5 • **Tensor Train Factorization (TTF)** enables distributed workload execution, ensuring parallel processing across multi-node architectures.
-

3. The system as claimed in claim 1, wherein the AI-governed workload
10 management module further comprises:

- **A dependency-aware workload allocation module**, employing **Graph Neural Networks (GNNs)** to:
 - Construct **task dependency graphs**, mapping execution interdependencies.
 - 15 ○ Detect cyclic dependencies, optimizing workload execution order dynamically.
 - **A multi-agent execution coordination model**, wherein:
 - **Federated reinforcement learning** refines workload execution models across distributed nodes.
 - 20 ○ **Policy adjustments incorporate energy efficiency constraints and real-time performance feedback.**
-

4. The system as claimed in claim 2, wherein the tensor-based execution control module further comprises:

- 25 • **A hierarchical tensor execution framework**, wherein:
- **Tensor segmentation dynamically adjusts execution blocks**, ensuring optimal processing granularity.
 - **Tensor weight optimization algorithms** modify execution coefficients based on:
 - 30 ■ **Power availability per processing node.**
 - **Predicted workload intensity using Gaussian Process Regression (GPR).**
-

5. A Byzantine Fault-Tolerant execution synchronization framework for
35 distributed computing, the system comprising:

- 5 • **A decentralized execution consensus protocol**, wherein:
- **Practical Byzantine Fault Tolerance (PBFT) is implemented via a three-phase commit protocol:**
 - **Pre-prepare phase:** Leader node proposes execution state.
 - **Prepare phase:** Validator nodes confirm execution integrity.
 - **Commit phase:** Final agreement is reached with cryptographic verification.
- 10
- **A leader election module**, employing:
- **A proof-of-stake model**, where execution nodes are ranked based on:
- 15
- **Execution reliability scores** derived from historical task completion rates.
 - **Computational efficiency benchmarks** for optimized workload assignment.
- 20
-
6. **The system as claimed in claim 5, further comprising an execution fault-resilient control engine, implementing:**
- **An AI-based failure detection module**, utilizing:
 - **Graph-based anomaly detection algorithms** to identify execution instability patterns.
 - **Preemptive failure mitigation mechanisms**, wherein:
 - Task execution is **restructured before failure occurrence**.
 - **An adaptive recovery and workload redistribution mechanism**, ensuring:
 - **Reallocation strategies leverage redundancy-aware execution balancing.**
 - **State restoration follows checkpoint-based rollback models.**
- 25
- 30
-

7. **The system as claimed in claim 5, further comprising:**

- 35 • **An execution latency minimization module**, wherein:

- 5
- **A reinforcement learning model trained on execution delay metrics** dynamically optimizes scheduling.
 - **A workload dependency verification engine**, ensuring:
 - **Consistency across execution clusters using multi-node version tracking algorithms.**

10

8. An AI-driven power optimization and execution scaling system, the system comprising:

- **A hierarchical Markov Decision Process (H-MDP)-based power optimization model**, wherein:

15

 - **Power state transitions dynamically adjust execution efficiency based on system constraints.**
- **A federated execution synchronization engine**, ensuring:
 - **Task execution is synchronized with real-time energy availability constraints.**

20

9. The system as claimed in claim 8, further comprising:

- **A real-time power state monitoring module**, wherein:
 - **A reinforcement learning-based adaptive power profiling algorithm** dynamically regulates:

25

 - **Energy consumption thresholds** per workload category.
 - **Execution reallocation strategies based on thermal efficiency models.**

30

10. The system as claimed in claim 8, wherein the execution load balancing module further comprises:

- **A workload execution queue optimizer**, dynamically adjusting execution order based on:
 - **Real-time congestion analysis via predictive queue modeling.**
- **An AI-driven execution timing controller**, ensuring:

- 5 ○ **Task prioritization dynamically aligns with power efficiency constraints.**

Title: System and Method for AI-Driven Energy-Efficient Workload Optimization in Distributed Computing Systems

10

6. DATE AND SIGNATURE:

15

20

25

30

35

5 **7. Abstract :**

The present invention provides a system and method for AI-driven energy-efficient workload optimization in distributed computing systems. The system employs an AI-governed workload scheduler that dynamically allocates tasks based on real-time power constraints, workload complexity, and reinforcement learning-based execution policies. A tensor-based execution module enables multi-dimensional workload decomposition, optimizing task execution paths. A multi-agent synchronization framework ensures coordinated workload execution across heterogeneous computing environments. The system further integrates an AI-driven energy recycling mechanism, converting computational heat dissipation into usable power via thermal capture models. Additionally, an adaptive execution migration module enables real-time workload redistribution based on power availability fluctuations and grid stability conditions. The invention achieves scalable, energy-efficient computing through AI-driven execution control, power-aware scheduling, and real-time computational resource scaling. The proposed method reduces execution latency and enhances workload adaptability in cloud computing, HPC, and edge AI environments.

10

15

20